

[5]
23
초

```
import builtins
print = builtins.print

name = "양영서"
tax_rate = 0.033
total_points = 0

print(f"--- {name}의 알바 급여 관리 시스템 ---")
print("현재 선택 가능한 작업 카테고리:")
print(categories)
print("-" * 40)

print("알파벳으로 작업 종류를 선택하세요 (A~G):")
choice = input("선택: ")

if choice == 'A':
    work_name = categories[0]
    ques_price = 2000
elif choice == 'B':
    work_name = categories[1]
    file_price = 500
    ques_price = 30
elif choice == 'C':
    work_name = categories[2]
    file_price = 400
    ques_price = 40
elif choice == 'D':
    work_name = categories[3]
    file_price = 300
    ques_price = 50
elif choice == 'E':
    work_name = categories[4]
    file_price = 200
    ques_price = 50
elif choice == 'F':
    work_name = categories[5]
    file_price = 70
    ques_price = 0
```



```
elif choice == 6:
    work_name = categories[6]
    file_price = 100
    ques_price = 0
else:
    print("잘못된 카테고리입니다. 프로그램을 다시 실행해 주세요.")
    work_name = "미선택"

if work_name != "미선택":
    f_count = int(input(f"작업한 파일 개수: "))
    q_count = int(input(f"작업한 문항 수: "))
    current_points = (f_count * file_price) + (q_count * ques_price)
    total_points += current_points
    tax_amount = total_points * tax_rate
    final_salary = total_points - tax_amount
```

```
print("\n" + "="*40)
print(f" [ 작업 보고서: {work_name} ]")
print(f"      누적 합계 금액: {total_points}원 ,")
print("-" * 40)
```

독립 if문 활용

```
if total_points > 0:
    print(f" 공제 세금(3.3%): {tax_amount:.1f}원")
    print(f" 최종 예상 수령액: {final_salary:.1f}원")

print("="*40)
```

```
def display_menu():
    print(f"--- {name}의 스마트 급여 매니저 V2.0 ---")
    print("1. 추가 작업 기록하기 (기존 데이터 유지)")
    print("2. 누적 정산 보고서 및 인센티브 확인")
    print("3. 매니저 프로그램 종료")
    print("-" * 40)
```

```
def get_incentive_rates(work_choice):
```

15]
23초



```
print("\n 입력한 작업에 대한 총합 출력")  
print("-" * 40)
```



```
def get_incentive_rates(work_choice):  
    if work_choice == 'A' or work_choice == 'B' or work_choice == 'C':  
        return [0.4, 0.02]  
    elif work_choice == 'D' or work_choice == 'E':  
        return [0.6, 0.03]  
    elif work_choice == 'F' or work_choice == 'G':  
        return [0.04, 0.0]  
    else:  
        return [0.0, 0.0]
```

5]
23초



```
def add_new_work():
```

```
    global total_points, total_incentive
```

```
    print("\n\n[추가 작업 기록 시스템]")  
    print("A~G 카테고리 중 하나를 입력하세요:")  
    new_choice = input("선택: ").upper()
```

```
    if new_choice == 'A':  
        f_price, q_price = 600, 20  
    elif new_choice == 'B':  
        f_price, q_price = 500, 30  
    elif new_choice == 'C':  
        f_price, q_price = 400, 40  
    elif new_choice == 'D':  
        f_price, q_price = 300, 50  
    elif new_choice == 'E':  
        f_price, q_price = 200, 50  
    elif new_choice == 'F':
```



13
초



```
elif new_choice == 'G':  
    f_price, q_price = 100, 0  
else:  
    print("잘못된 카테고리입니다. 메뉴로 돌아갑니다.")  
    return
```

```
new_f = int(input("추가 작업한 파일 개수: "))  
new_q = int(input("추가 작업한 문항 수: "))
```

```
work_pay = (new_f * f_price) + (new_q * q_price)
```

```
inc_rates = get_incentive_rates(new_choice)  
work_inc = (new_f * inc_rates[0]) + (new_q * inc_rates[1])
```

```
total_points += work_pay  
total_incentive += work_inc
```

```
print(f"성공적으로 누적되었습니다 (추가 급여: {work_pay}원 / 인센티브: {work_inc:.2f}P)")
```

```
def print_final_report():  
    print("\n\n" + "=" * 40)  
    print(f" [ {name}의 누적 정산 및 인센티브 보고서 ] ")  
    print(f" • 누적 기본 급여 합계: {total_points:,}원")  
    print(f" • 누적 인센티브 포인트: {total_incentive:.2f}P")  
    print("-" * 40)
```

```
bonus_pay = (int(total_incentive) // 50) * 10000
```

```
grand_total = total_points + bonus_pay
```



3
초





```
final_tax = grand_total * tax_rate
final_receipt = grand_total - final_tax

if total_incentive >= 50:
    print(f"축하합니다! 인센티브 {int(total_incentive)}P를 달성하여")
    print(f" 보너스 수당 {bonus_pay:,}원이 추가 지급됩니다!")
else:
    print(f" 보너스까지 남은 인센티브: {50 - total_incentive:.2f}P (50P당 1만원)")

print("-" * 40)
print(f" 총 세전 금액 (보너스 포함): {grand_total:,}원")
print(f" 공제 세금 (3.3%): {final_tax:.1f}원")
print(f" 최종 실수령 예상액: {final_receipt:.1f}원")
print("=" * 40)
```

```
total_incentive = 0
```

51
23
초



```
while True:
    display_menu()
    menu_choice = input("원하는 메뉴 번호를 입력하세요: ")

    if menu_choice == '1':
        add_new_work()
    elif menu_choice == '2':
        print_final_report()
    elif menu_choice == '3':
        print(f"{name} 매니저 프로그램을 안전하게 종료합니다. 수고하셨습니다!")
        break
    else:
        print("잘못된 번호입니다. 1~3번 메뉴 중에서 선택해 주세요.")
        continue
```

--- 양영서의 알바 급여 관리 시스템 ---
현재 선택 가능한 작업 카테고리:
... ['자료 입력', '문제 검토', '데이터 분류', '문서 정리', '번역 작업', '단순 반복 작업', '사무 보조']

알파벳으로 작업 종류를 선택하세요 (A~G):

선택: A

작업한 파일 개수: 9

작업한 문항 수: 389

=====

[작업 보고서: 자료 입력]

- 이번 작업 급여: 13180원

- 누적 합계 급여: 13180원

=====

공제 세금(3.3%): 434.9원

최종 예상 수령액: 12745.1원

=====

--- 양영서의 스마트 급여 매니저 V2.0 ---

1. 추가 작업 기록하기 (기존 데이터 유지)

2. 누적 정산 보고서 및 인센티브 확인

3. 매니저 프로그램 종료

--- 양영서의 스마트 급여 매니저 V2.0 ---

1. 추가 작업 기록하기 (기존 데이터 유지)

... 2. 누적 정산 보고서 및 인센티브 확인

3. 매니저 프로그램 종료

원하는 메뉴 번호를 입력하세요: 1

[추가 작업 기록 시스템]

A~G 카테고리 중 하나를 입력하세요:

선택: G

추가 작업한 파일 개수: 27

추가 작업한 문항 수: 0

성공적으로 누적되었습니다 (추가 급여: 2700원 / 인센티브: 1.08P)

--- 양영서의 스마트 급여 매니저 V2.0 ---

1. 추가 작업 기록하기 (기존 데이터 유지)

2. 누적 정산 보고서 및 인센티브 확인

3. 매니저 프로그램 종료

원하는 메뉴 번호를 입력하세요: 2

=====

[양영서의 누적 정산 및 인센티브 보고서]

원하는 메뉴 번호를 입력하세요: 2



=====

[양영서의 누적 정산 및 인센티브 보고서]

- 누적 기본 급여 합계: 15,880원
- 누적 인센티브 포인트: 1.08P

보너스까지 남은 인센티브: 48.92P (50P당 1만원)

총 세전 금액 (보너스 포함): 15,880원
공제 세금 (3.3%): 524.0원
최종 실수령 예상액: 15356.0원

--- 양영서의 스마트 급여 매니저 V2.0 ---

1. 추가 작업 기록하기 (기존 데이터 유지)
2. 누적 정산 보고서 및 인센티브 확인
3. 매니저 프로그램 종료

원하는 메뉴 번호를 입력하세요: 3

양영서 매니저 프로그램을 안전하게 종료합니다. 수고하셨습니다!